AI 전문가 양성을 위한「 KB AI 엘리트 교육」과정

TxLens

마이데이터 거래내역 정규화 · 분류 + 패턴 탐지 에이전트

> " 가맹점명을 못 묶으면 구독도 못 찾는다 "



6 주차 최종 발표 · 2026-10-17 (토)

한눈에 보기

TxLens: 거래내역 정규화 · 분류 + 패턴 탐지

- 제각각인 가맹점명 정리 + 카테고리 분류 → 그 위에서 구독 · 이상지출 · 이상거래 탐지
- LangChain 1.0 `create_agent` · LangGraph · Pydantic v2 · Streamlit
- 1 부 ·2 부 · 화면 4 개 · 사람 교정까지 동작 확인 (테스트 384 개)
- 정확도 : 룰만 93% / LLM 만 86% / 하이브리드 99% → 예상과 순서가 달랐음

해결하려는 현업 문제

아래 여섯 줄 , 전부 같은 가맹점

```
  ( 주 ) 스타벅스커피코리아 1234 호점
  STARBUCKS  1234
스타벅스 강남역점
  NAVERPAY* 스타벅스
  ( 주 ) 에스씨케이컴퍼니      <-  법인명만 온다 .  브랜드를 알 수 없다
  ( 주 ) 케이지이니시스        <-  PG  경유 .  실제 가맹점이 사라졌다
```

	내용
누가	지출분석 화면을 보는 고객 · 룰 사전을 관리하는 담당자
언제	거래내역이 들어올 때마다 (신규 브랜드 · 지점은 매일 생김)
왜 생기나	표준 API 가 필드 구조만 정하고 가맹점명은 사실상 자유 텍스트
결과 1	카테고리 오분류 → 지출분석 화면을 못 믿게 되고 안 쓰게 됨
결과 2	' 기타 ' 증가 → 분석 가치 하락 , CS 문의 증가
결과 3	사전 수작업 관리 → 새 표기가 계속 들어와 끝이 안 남

왜 룰만으로는 안 되는가

하나 . 롱테일이라 , 같은 룰도 세는 기준에 따라 93% vs 42%

룰 사전 v1 (head 29 개) 로 측정	거래 건수 기준	정규화 키 기준
L2 정확도	93.0%	42.0%
미매칭	7.7%	58.7% (키 264 개 중 155 개)

둘 . 표기 편차가 카드사 · 페이사별로 뭉쳐 있음 (실무 확인)

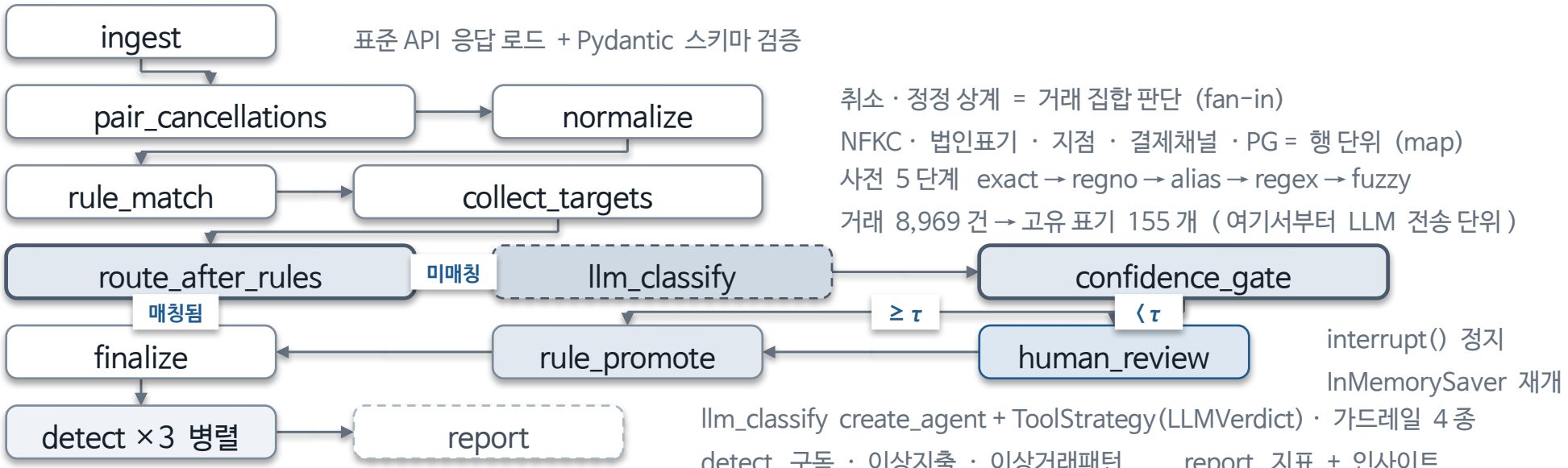
기관	비중	regno 채움	지배적 표기	난이도
A 카드	30%	95%	브랜드명 · 지점명	쉬움
B 카드	25%	50%	법인명 (브랜드 불명)	어려움
C 카드	20%	20%	PG 노출 25%	매우 어려움
D 페이	15%	35%	간편결제 프리픽스 70%	중간
E 카드	10%	10%	전부 섞임 , 오타 10%	매우 어려움

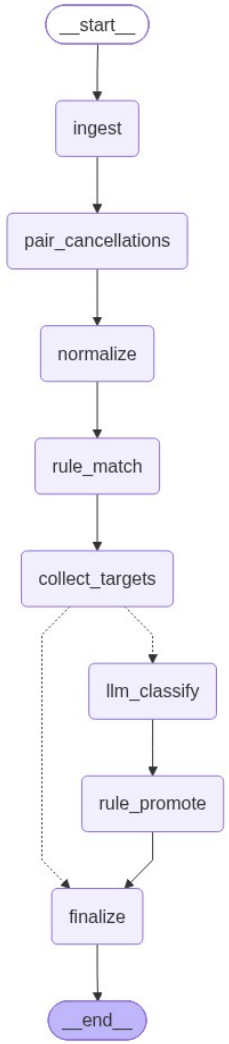
무엇을 하는 에이전트인가

1 부가 이름을 묶어야 2 부가 찾을 수 있음

	1 부 : 정규화 · 분류	2 부 : 패턴 탐지
입력	표준 API 형태 거래내역	1 부의 출력
방법	룰 (head) + LLM (tail) + 사람 교정 → 사전에 반영	통계 규칙 (LLM 안 씀)
출력	`merchant_canonical` · L1/L2 카테고리 · 신뢰도	구독 목록 · 이상지출 알림 · 이상거래 알림
지표	정확도 · 미분류율 · LLM 호출 감소	precision / recall

기능 흐름도





실제 그래프 (C)

- 설계 노드 13 개 중 LLM 이 들어가는 건 `llm_classify` 하나. 나머지는 테스트로 고정
- 조건부 분기 2 개 (`route_after_rules`·`confidence_gate`), 사람 개입 1 개, 사전으로 돌아오는 루프 1 개
- 지금 실제로 컴파일되는 노드는 8 개 (`rule_promote` 포함)
- `build_graph(review=True)` 로 켜면 승격 자리에 사람 검토 서브그래프가 들어감
- 탐지 3 종은 그래프 노드가 아닌 일반 함수 (`finalize` 결과를 받음)
  - 오른쪽 그림 : `get_graph()` 출력 그대로
- **A/B/C 세 구성 모두 이 그래프 하나로 돎.** 바꾸는 건 사전과 분류기뿐

State 설계 : 스키마는 TypedDict, 필드는 Pydantic

	선택	이유
상태 스키마 `TxState`	`TypedDict (total=False)`	노드는 바뀐 키만 반환 . Pydantic 을 스키마로 쓰면 갱신 때마다 전체 재검증 (8,969 건 × 노드 8 개)
필드 타입	Pydantic `BaseModel`	단계 간 계약 : `RawTx` → `NormalizedTx` → `ClassifiedTx`. 단계마다 보장하는 게 타입에 드러남

reducer 는 필드 성질에 맞춰 선택

성질	필드	reducer
다음 단계가 대체한다	`raw·pairings·normalized·matched·confirmed·metrics`	없음 (교체)
여러 노드 · 라운드가 덧붙인다	`review_queue·human_corrections·rule_proposals·llm_failed`	`operator.add`
표기 단위 결과를 합친다	`llm_results`	`merge_verdicts` (나중 값 우선)

- `Annotated[..., operator.add]` 는 편하지만 값이 계속 쌓이는 점 주의
- `llm_results` 키 = 정규화 표기 = 캐시 키
- `human_corrections` 는 쓰는 쪽과 읽는 쪽이 둘 다 생긴 뒤에 추가 (`insights` 가 아직 없는 이유)
- 검토 서브그래프는 **별도 스키마** (`ReviewState`). 거래 데이터를 안 들고 감

Node · Edge 설계 : 어디서 나눌까

분리한 노드	근거
`preprocess` → `pair_cancellations` + `normalize`	앞은 (기관, 고객, 승인번호)로 묶어 합계를 보는 fan-in, 뒤는 행 단위 map
`rule_match` → `rule_match` + `collect_targets`	처리 단위가 거래 → 고유 표기로 바뀌는 지점. 여기서 LLM 비용이 정해짐

Edge: 조건부 분기 2 개 + 일부러 안 만든 분기

- `route_after_rules`: 미매칭이 없으면 `llm_classify` 건너뜀
- `confidence_gate`: 신뢰도 τ 미만만 `human_review` 로. **볼 게 없으면 사람을 안 부름**
- (A) 는 `classifier=None` 이라 그 노드와 분기 자체를 안 만듦
- Send 로 표기별 병렬 처리는 안 함 (호출 수를 결정론으로 유지)

llm_classify 노드 안 : 가드레일도 노드

```
__start__
    |
PIIMiddleware[credit_card].before_model  --(차단)--> __end__
PIIMiddleware[customer_id].before_model  --(차단)--> __end__
ModelCallLimitMiddleware.before_model    --(상한)--> __end__
    |
  model  ---->  ModelCallLimitMiddleware.after_model
                        |
                ToolCallLimitMiddleware.after_model  --(상한)--> __end__
                        |
                PIIMiddleware[*].after_model
                   |          |
                 tools     __end__  (structured_response = LLMVerdict)
                   |
                   +---> 다시 before_model 로 (ReAct 루프)
```

- 가드레일 4종이 그래프 노드로 들어감 : `PIIMiddleware` 2종 · `ModelCallLimit` · `ToolCallLimit`
- `model` ↔ `tools` 루프 . 표기 155개에 모델 호출 587회
- **개인정보 차단은 프롬프트가 아닌 미들웨어에서**
- 위 그림 : `agent.get_graph()` 출력 그대로

Agent 입출력 : 도구 3 개 + structured output

입력 : 사전을 프롬프트에 다 넣지 않고 도구로 조회

도구	하는 일	왜 도구인가
`lookup_alias`	별칭 사전 유사 조회 (`token_sort_ratio` ≥ 55)	사전이 커져도 프롬프트는 그대로
`search_taxonomy`	택소노미에서 후보 카테고리 검색	호출하면 더 정확 (을지로호프)
`get_brand_hint`	전처리 산출물 조회	이미 뽑은 값은 다시 추론 안 시킴

출력 : response_format=ToolStrategy(LLMVerdict) 한 줄

막으려는 값	막는 장치
택소노미에 없는 카테고리	`CategoryL1·CategoryL2` Enum (자유 문자열 불가)
`l1=food` + `l2=cafe.coffee`	`model_validator` (실제로 나왔던 조합)
`confidence=1.7`	`Field(ge=0, le=1)`
근거 없는 판정	`reasoning·evidence` 필수 (검토자가 근거를 봐야 함)

도구를 밖으로 : 사람과 에이전트가 같은 도구를 씀

내보낸 것 : python -m txlens.mcp_server (stdio, 외부 네트워크 없음)

종류	무엇	쓰는 쪽
에이전트 도구 3 종	`lookup_alias` · `search_taxonomy` · `get_brand_hint`	에이전트와 사람
사람용 도구 2 종	`classify_merchant` · `dictionary_status`	사람만
리소스 2 종	`txlens://taxonomy` · `txlens://dictionary`	붙여 읽는 자료

도구는 한 벌만 유지

- 기존 `make_tools()` 를 감싸기만 함 . 도구 설명도 같은 걸 쓰고 , 둘이 같은지 테스트로 확인
- 실행 설정은 `.mcp.json` 한 곳 . Claude Code 와 우리 클라이언트가 같은 서버에 붙음

기본 실행 경로는 그대로 (전달 방식 때문에 수치가 바뀌면 안 되므로)

- `llm_classify` 는 in-process 그대로 . MCP 경유로 바꾸면 그래프를 async 로 바꾸고 A/B/C 를 다시 재야 함
- 대신 `mcp_server/client.py` 로 **두 경로의 답이 같은지** 확인만 해 둠

기술 스택 : 배운 걸 어디에 썼나

배운 기술 (LangChain 1.0 이후)	TxLens 에서 쓰는 자리	지금
`create_agent`	`llm_classify` 노드 안 (tail 분류만)	실측
`ToolStrategy(LLMVerdict)`	분류 출력 스키마	실측
`PIIMiddleware(strategy="block")` ×2	모델 호출 직전 차단	테스트
`ToolCallLimit·ModelCallLimit`	실행당 상한 `run_limit`	테스트
`AgentMiddleware.wrap_model_call`	호출 캐시 (히트면 모델 안 부름)	테스트
`InMemorySaver` / `InMemoryStore`	`human_review` 재개 / 룰 사전	실사용
`LLMToolEmulator` + 스크립트된 모델	API 키 없이 그래프 경로 · 툴 콜 순서 고정	384 tests
MCP `FastMCP` (stdio 서버)	에이전트 도구를 프로세스 밖으로	실사용
`langchain_mcp_adapters`	같은 서버에 우리 코드로도 붙어 답을 대조	테스트

세 가지 구성 비교 : 예상이 빗나간 곳

구성	거래 L2	키 L2	미매칭	확정 커버리지	LLM 키 / 모델 호출
(A) 룰만	93.0%	42.0%	7.7%	84.3%	0 / 0
(B) LLM 만 (사전 없음)	85.6%	66.7%	5.1%	86.8%	257 / 1,026
(C) 하이브리드	98.9%	86.4%	1.6%	90.4%	155 / 587

- 예상 : LLM 만 쓰는 B 가 제일 정확 → 실제로는 **룰만 쓴 A 보다 7%p 낮음**
- B 는 답을 많이 내지만 틀린 답도 자신 있게 냄 (쇼핑 500 건을 식비로 분류)
- 같은 모델 · 프롬프트라도 하이브리드 안에서는 LLM 정확도 88% → 95%
- PG 사 경유로 가맹점명이 지워진 8% 는 어떤 구성으로도 못 품

사전 승격 루프 : 자동은 별칭까지 , 새 브랜드는 사람이

라운드	사전 키	LLM 표기	모델 호출	거래 L2	키 L2	자동 승격
1 · v1	65	155	572	98.0%	87.9%	35 건
2 · v2	100	119	438	98.4%	90.5%	8 건
3 · v3	108	111	394	98.1%	91.7%	1 건
4 · v4	115	104	377	98.0%	90.2%	6 건
5 · v5	121	98	351	97.7%	90.2%	3 건

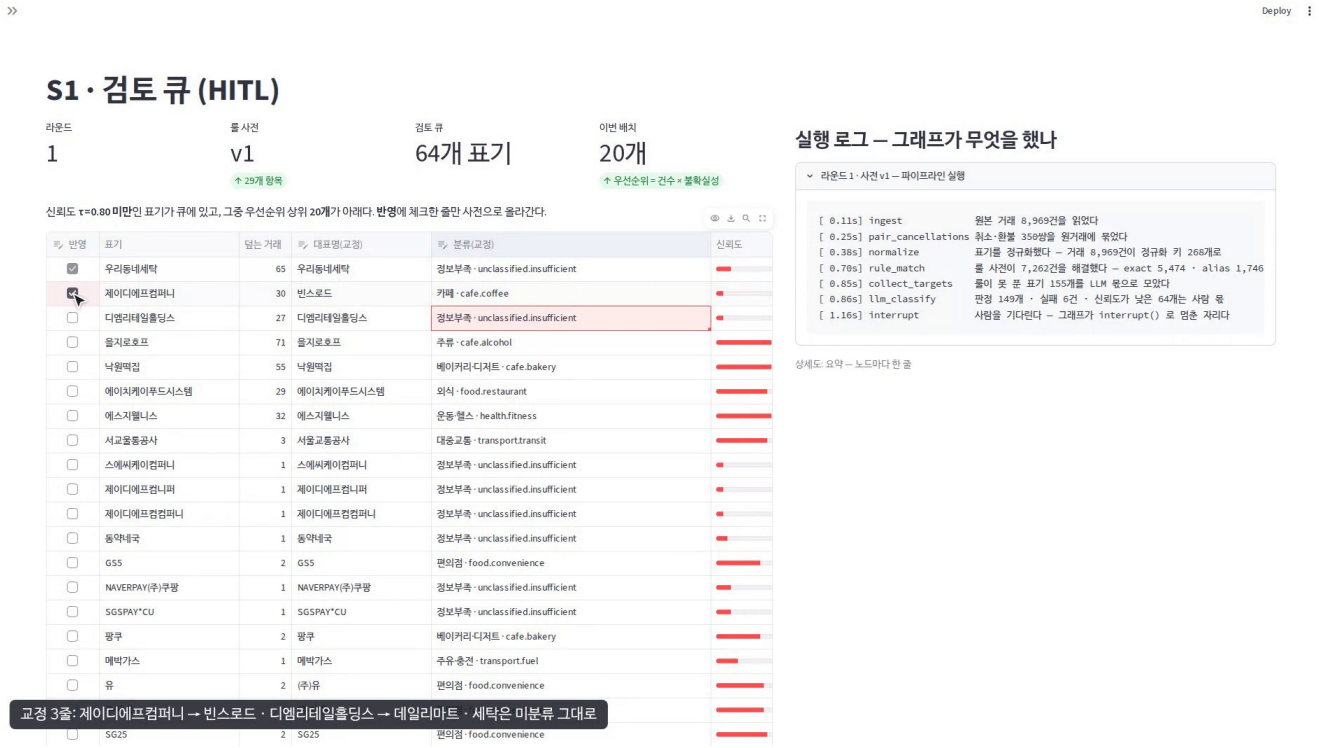
- 판정 근거별 정확도를 정답셋으로 측정 : 브랜드 인식 96% · 키워드 79% · 추측 9%
- 진짜 위험은 대표명 . 타벅스가 새 브랜드로 굳으면 스타벅스가 둘로 쪼개짐 → 자동 승격은 아는 브랜드의 별칭만
- **5 라운드 내내 수렴하지 않음** . 오타가 계속 새로 나옴 (GS2·GS5·GS52·GSS25…)
- 승격 판정은 거의 다 맞음 (1 라운드 40 건 중 39 건 · 4~5 라운드 13 건 전부). 호출 37% 감소 , 멈출 시점은 사람이 판단
- 검토 화면 (S1) 에서 세 줄 고치면 사전 29 → 32, 거래 122 건 바로 반영 , 다음 라운드 표기 155 → 115

2 부 탐지 : 분류가 좋아지면 탐지도 는다

분류 단계	미묶음	구독 P/R	이상지출 P/R	이상거래 P/R
정규화만 (사전 ·LLM 없음)	100%	95.3 / **56.2**	— / 0.0	96.4 / 79.4
(A) 룰만	15.7%	94.7 / 74.0	63.2 / 61.0	97.1 / 100.0
(C) 하이브리드	9.6%	94.7 / 74.0	62.5 / 67.8	97.1 / 100.0
정답 분류 (상한)	8.0%	95.7 / **90.4**	62.7 / 71.2	97.1 / 100.0

- 이름을 못 묶으면 구독은 절반 (56%) 만 잡힘 . 넷플릭스가 표기 3 개로 쪼개지는 식
- 이상지출은 0%, 이상거래는 79% 유지 . 카테고리에 기대는 정도 차이
- 남은 구독 손실 (74% → 90%) 은 대부분 **사전이 잘못 묶은 것** (쿠팡와우 → 쿠팡)
- 일부러 심은 오탐 함정 60 건 중 통과 0 건
- 고객당 알림 월 0.6 건 · 급증 정밀도 37% 는 보수적으로 잡힌 값

화면으로 확인한 것



S1 검토 큐 · 왼쪽이 사람이 고치는 자리 , 오른쪽이 8 노드 실행 로그

- 영상 네 구간 : S1 교정 (1:08) → S4 토글 (0:36) → S2 지표 (0:24) → 준비 상태 (0:08)
- 오른쪽 칸 로그가 그래프가 도는 증거 — 지연을 뺀 실제 노드 시간
- 라이브 아닌 녹화 · 스틸도 같은 영상에서 뽑아 덱과 어긋나지 않음
- 저장된 LLM 판정을 재생 — 데모 중 외부 호출 0

분류 결과를 어디에 남기나

지금은 메모리에만 있음

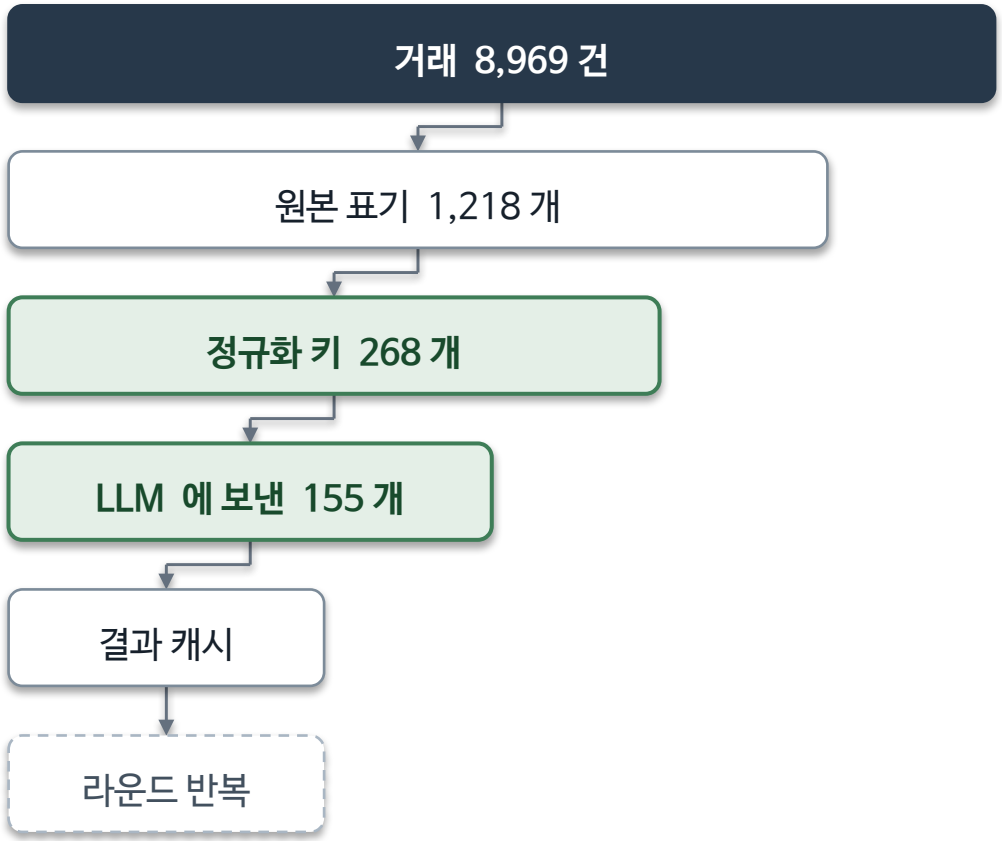
- 분류 결과 8,619 건은 그래프 상태에만 있고 프로세스가 끝나면 사라짐
- 파일로 남는 건 집계 지표 (`eval --json`) 뿐
- **일부러 이렇게 함 :** 시드만 같으면 재현되고 이전 실행의 영향을 안 받음

대신 남기는 것

무엇	어디에	왜
룰 사전 v{n}	`InMemoryStore` namespace `("rules","v{n}")` + `rules/v{n}.yaml`	승격 루프 결과 . 버전 관리 · 롤백 · 감사 추적 필요
검토 큐와 스레드 상태	`InMemorySaver` (checkpointer)	`interrupt()` 로 멈춘 곳에서 이어서 진행 (검토 화면에서 실제 사용)
거래별 분류 결과	남기지 않는다	사전이 원본이고 분류 결과는 파생물 . 사전이 바뀌면 저장한 결과는 바로 낡음

- 화면 (Streamlit) 도 결과를 세션 안에서만 보관
- 라운드마다 바뀌는 값은 캐시하지 않음
- 남은 과제 : 운영에서 매번 전량 재분류 (약 33 분) 는 무리 → 사전 버전 기준 결과 캐시 검토
- 덤으로 거래 데이터가 파일에 안 남음 (규제 측면에서도 유리)

기대 효과



합성 실측 · 고객 30 명 × 6 개월 · 시드 42

전처리 전 · 거래 대비 13.6%

전처리 후 · 실제 LLM 전송 단위 · 거래 대비 3.0%

룰 5 단계가 head 를 덮고 남은 것 · 하이브리드 (C) 실측 · 거래 대비 1.7%

같은 키는 다시 보내지 않음 (히트는 승격 루프 2 회차부터)

사전 성숙 → 외부 호출 0 에 수렴 → 온프레미스 / sLLM 이전

- 비용 : 외부로 나가는 건 전체 거래의 1.7%
- 운영 : 사전 관리가 상시 수작업 → 승격 결과 검토로
- 실무 도입 : 외부로 나가는 건 **가맹점명 문자열뿐** (미들웨어로 강제)

개발 과정 : 1 주차부터 오늘 (6 주차) 까지

1~2 주차 [완료]

주제 확정 · 기획 · 데이터 기반
블루프린트 · 택소노미 · 스키마 · 제공기관 프로파일 · 생성기 · 정답셋

3 주차 [완료]

결정론 파이프라인 + 측정 (중간 발표 · 9/5)
ingest · preprocess (표기 1,218 → 키 268) · rule_match 5 단계 · 평가 하네스
발표한 것 : " 룰만으로는 거래 93.0% · 키 42.0%"

4 주차 [완료]

LangChain 1.0 Agent + 가드레일 + 메모리 + 승격 루프 + 2 부 탐지
create_agent + ToolStrategy · 가드레일 4 개 · (B) (C) 실측 · 실호출 5 라운드 승격 53 건
발표한 것 : 가설 기각 · 승격 루프에 고정점 없음 · 2 부 전이

5 주차 [완료]

사람 교정 (HITL) + 화면 + MCP · 여기서 발표 가능한 완성형
interrupt · human_review → 승격 · S1 교정 세 줄 → 거래 122 건 즉시 반영
S2 는 그 자리에서 다시 잰다 (API 0) · 도구를 MCP stdio 서버로 프로세스 밖에

6 주차 [오늘]

최종 발표 10/17 (10 분 + Q&A · 데모 필수)
승격 루프에 고정점이 없음 , 멈추는 건 사람 판단 · 데모는 화면 녹화로

컷 순서

미리 정해 둔 컷 순서 : 모델 2 단 escalation → S3 거래 탐색기 → 이상거래 후순위 → 골드셋 축소
일정이 안 밀려서 안 씀 . S1 · S2 · S4 · 탐지 3 종 · 룰 5 단계 · 전이 실험 전부 들어감